

University
Politehnica
of Bucharest





Search Engines (II)
Apache Lucene

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro



Overview

- Apache Lucene
- Apache Lucene Architecture



Overview

- Apache Lucene
- Apache Lucene Architecture



Apache Lucene

- Apache Lucene™ is an open-source search software project
- The project releases are
  - Lucene™ core - the core search library written in Java
  - PyLucene - a python binding for Lucene.
- Lucene Core is a Java library providing
  - Powerful indexing
  - Search features
  - Spellchecking
  - Highlighting
  - Advanced analysis/tokenization capabilities
- PyLucene provides Python bindings for Lucene Core.



Apache Lucene

- Applications that can be developed using Apache Lucene:
  - Document analytics: Loading and traversing text documents based on some given criteria, finding the top terms from documents, aggregating on specific fields.
  - Log analytics: Analyzing application logs with high-performance dashboards built directly on top of Lucene.
  - Geospatial search: With the recent advent of geospatial queries and data structures, Lucene is fast becoming a popular choice for indexing latitudinal and longitudinal data and queries such as colocation searches.
  - Vector Database – improvements added by Elastic.co to make Elasticsearch a retrieval platform for Retrieval Augmented Systems (RAG)

- Two major search-based application have spawn from Lucene:
  - Apache Solr
  - Elasticsearch



Apache Lucene Features

- Text/Language Analysis
  - The analysis is used to create appropriate internal representations for
    - Document to be indexed
    - Queries for searching documents
  - During indexing, tokens are created and stored in Lucene's Inverted Index
  - At query time, tokens are created to form appropriate query representations



Apache Lucene Features

- Text/Language Analysis
  - The analysis consists in the following 3 steps:
    - Optional character filtering and normalization (e.g., removing diacritics)
    - Tokenization
    - Token filtering (e.g., stemming, lemmatization, stopword removal, n-gram creation)
  - This feature is used by Lucene for modeling documents



Apache Lucene Features

- Indexing and Storage
  - Used to index and store user defined documents
  - High degree of concurrency given by the lock-free indexing
  - Near real time indexing: documents become searchable as soon as they are indexed
  - Transactional support for additions and rollbacks
  - Different scoring models for terms, documents, and data sets

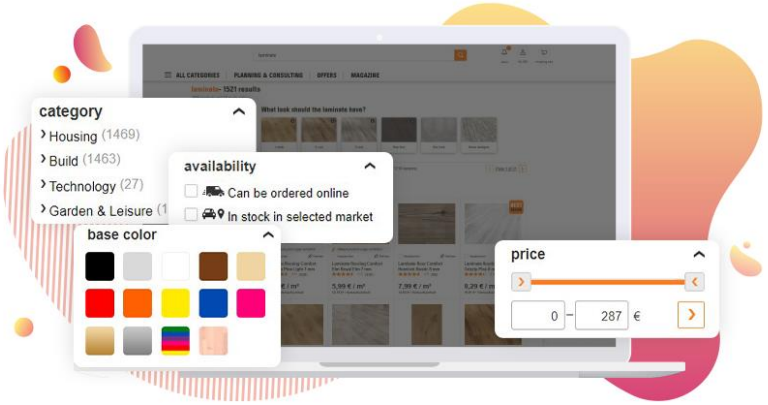


Apache Lucene Features

- Querying
  - Query options include the ability to filter, paginate, sort results, and perform relevance feedback
  - Multiple query representations, query parsers, and a query parsing framework for developer defined query parsers
  - Pluggable scoring model system for developers to implement their own
  - Predefined scoring models: traditional vector-space scoring model, Okapi BM25, Language Modeling, etc.

Apache Lucene Features

- Developer libraries
  - Used for building search-based applications
  - Example libraries:
    - Result highlighting (snippet generation)
    - Faceting (show part of the analysis result) and Faceted Search (technique that involves augmenting traditional search techniques with a faceted navigation system)
    - Spatial search
    - Document grouping by key (e.g., group all documents with the same base URL together)
    - Document routing (via an optimized, in-memory, single document index)
    - Point-based spatial search
    - Auto-suggest

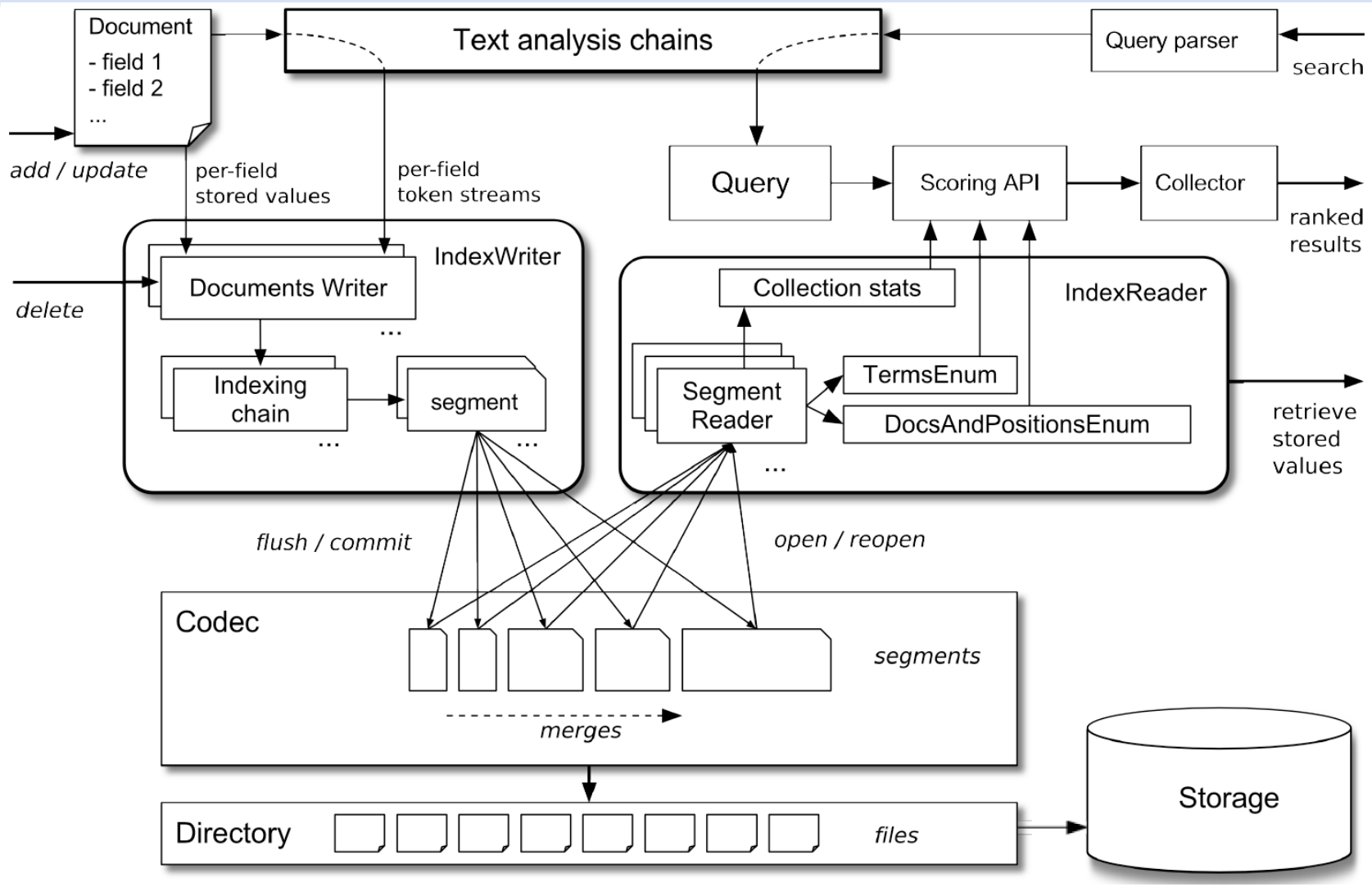




Overview

- Apache Lucene
- Apache Lucene Architecture

Apache Lucene Architecture





Apache Lucene Architecture

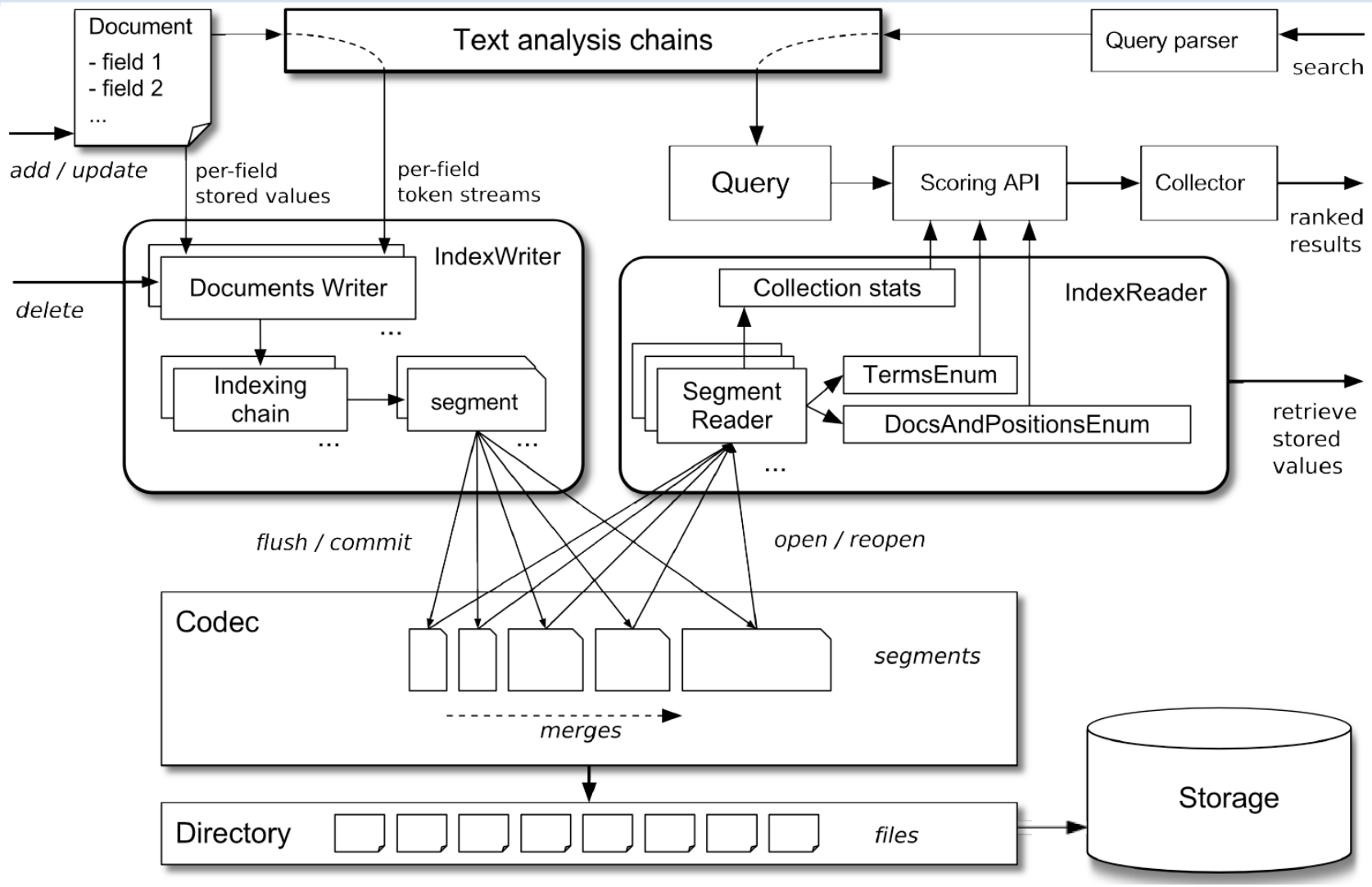
- Text Analysis
  - The text analysis chain produces a stream of tokens from the input data
  - Tokens are represented as a collection of attributes:
    - Term
    - Token position
    - Starting and ending offsets
    - Token type
    - Arbitrary payload data (a byte array to be stored in the index at the current position)
    - Integer flags
    - Other custom application-defined attributes (e.g., part-of-speech tags)



Apache Lucene Architecture

- Text Analysis
  - The text analysis chains consist of
    - Character filters (e.g., diacritics striping)
    - Tokenizers (which are the sources of token streams)
    - Series of token filters that modify the original token stream
  - Token streams perform specific functions such:
    - Tokenization by patterns, rules, and dictionaries (e.g., whitespace, regex, Chinese / Japanese / Korean)
    - Specialized token filters for efficient indexing of numeric values and dates (to support numerical range searching)
    - Language-specific stemming and stop word removal
    - Creation of character or word-level n-grams
    - Tagging, etc.

Apache Lucene Architecture





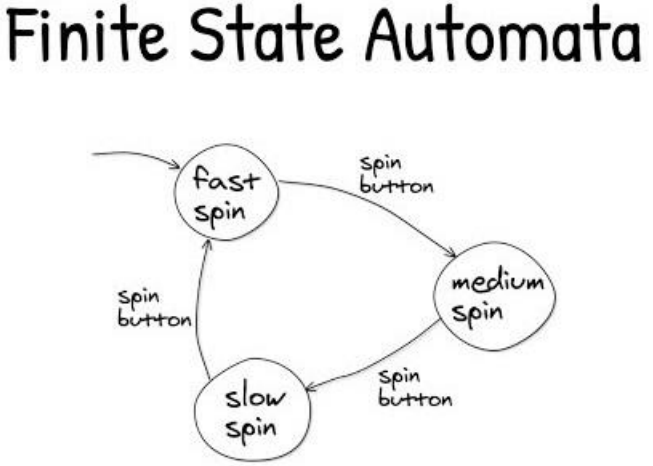
Apache Lucene Indexing Process

- Indexing
  - Lucene uses the well-known inverted index representation, with additional functionality for keeping adjacent non-inverted data on a per-document basis.
  - Both in-use and persistent data uses a variety of encoding schemas that affect the size of the indexed data and the cost of the data compression
  - Lucene uses pluggable mechanisms for data encoding (i.e., Codec API) and for the actual storage of index data (i.e., Directory API).
  - Incremental updates are supported and stored in index extents (i.e., segments) that are periodically merged into larger segments to minimize the total number of index parts



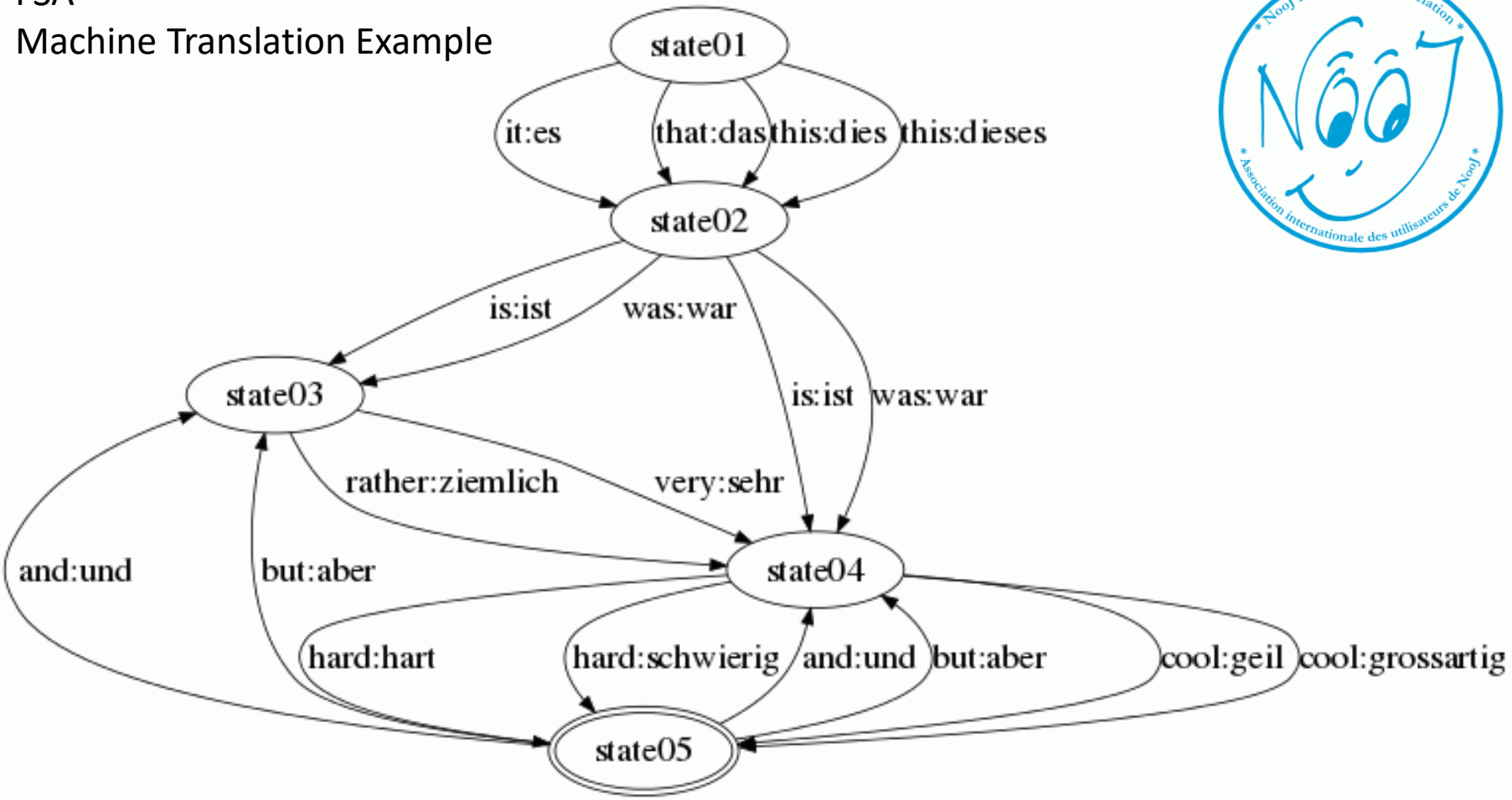
Apache Lucene Indexing Process

- Finite State Automata
  - The in-memory portion of the inverted index is implemented using a Finite State Transducer (FST)
  - An FST is a type of Finite State Automaton (FSA) that maps between two sets of symbols using two memory tapes:
    - Read a set of strings on the input tape
    - Generates a set of relations on the output tape



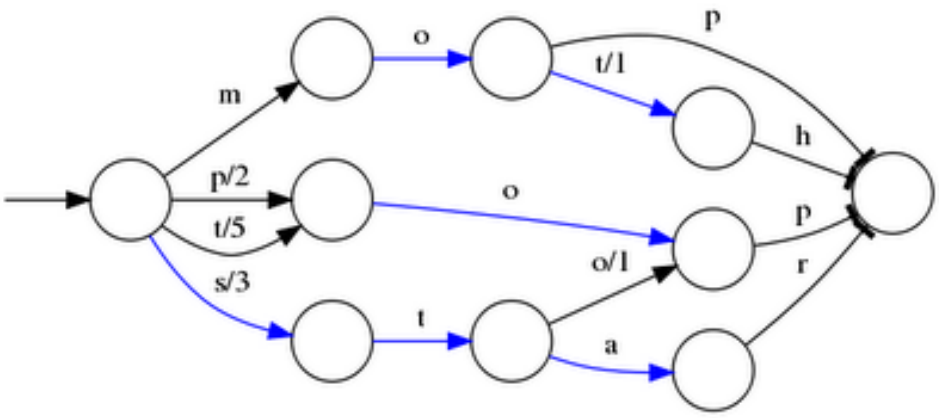
Apache Lucene
Indexing Process

FSA
Machine Translation Example



Apache Lucene Architecture

- The Finite State Transducer (FST) supports
  - Linear time construction of the minimal automaton
  - Compression
  - Reverse lookups
  - Weighted automata (used in NLP to assign weights to words and sentences)
  - Pluggable output algebras
  - Synonym processing
  - Spell correction
  - Auto-suggest/complete
  - Machine translation





Apache Lucene Indexing Process

- Document Model
  - Documents
    - Are modeled in Lucene as a flat ordered list of fields with content
    - Are not required to have a unique identifier (though they often carry a field with this role for application-level unique key lookup)
    - During the indexing process, documents are assigned internal integer identifiers
  - Fields have a name, content data, a float weight (used later for scoring), and other attributes, depending on their type, which together determine how the content is processed and represented in the index
  - There can be multiple fields with the same name in a document, in which case they will be processed sequentially



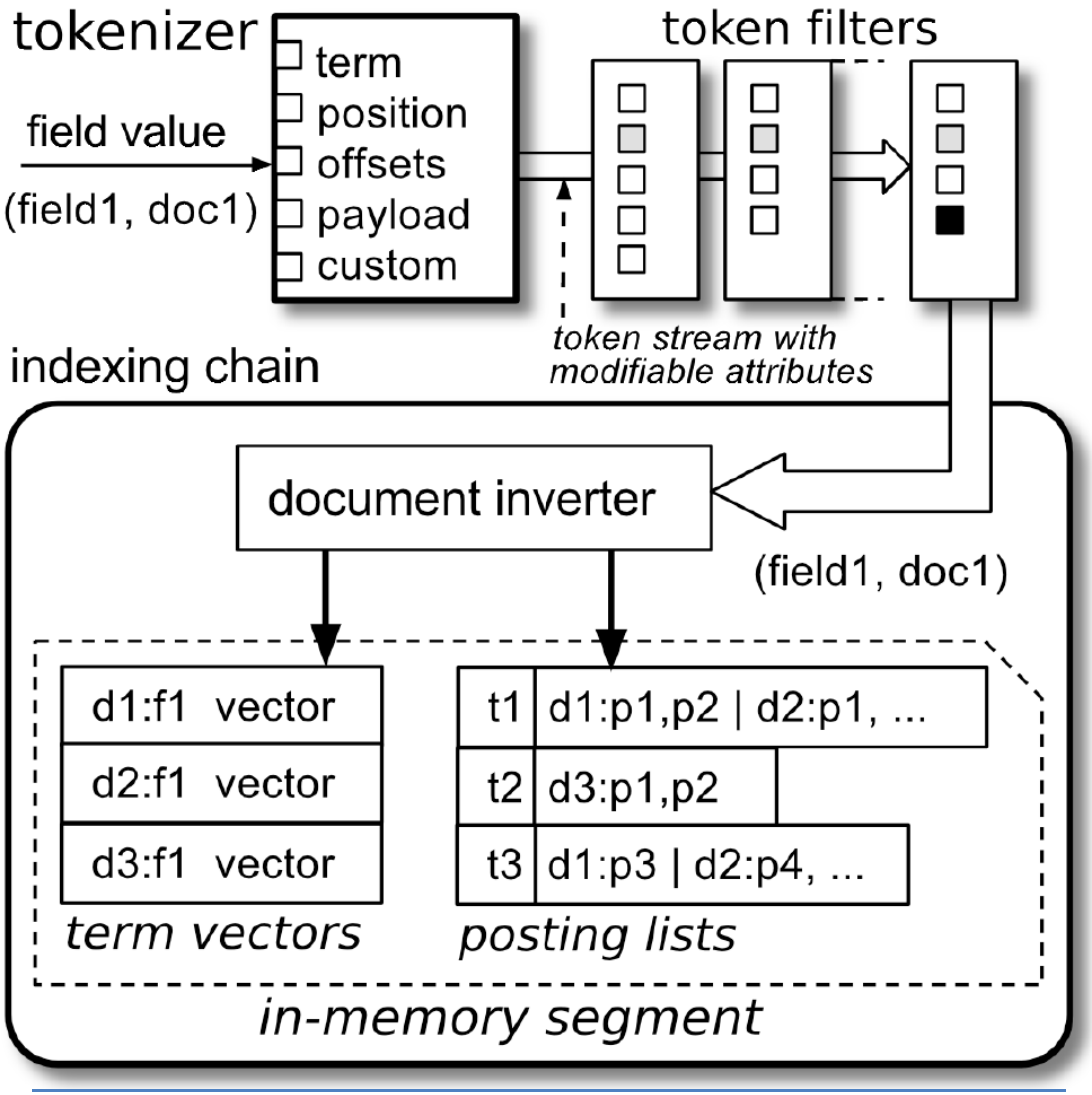
Apache Lucene Indexing Process

- Field Types
  - There are two broad categories of fields in Lucene documents
  - Indexed fields contain content to be inverted
  - Stored fields contain content to be stored

- Fields may belong to either or both categories (e.g., with content both to be stored and inverted).

- Both fields can be submitted for storing / indexing
  - Only stored fields can be retrieved
  - Indexed/Inverted data can be accessed and traversed

Apache Lucene Indexing Process

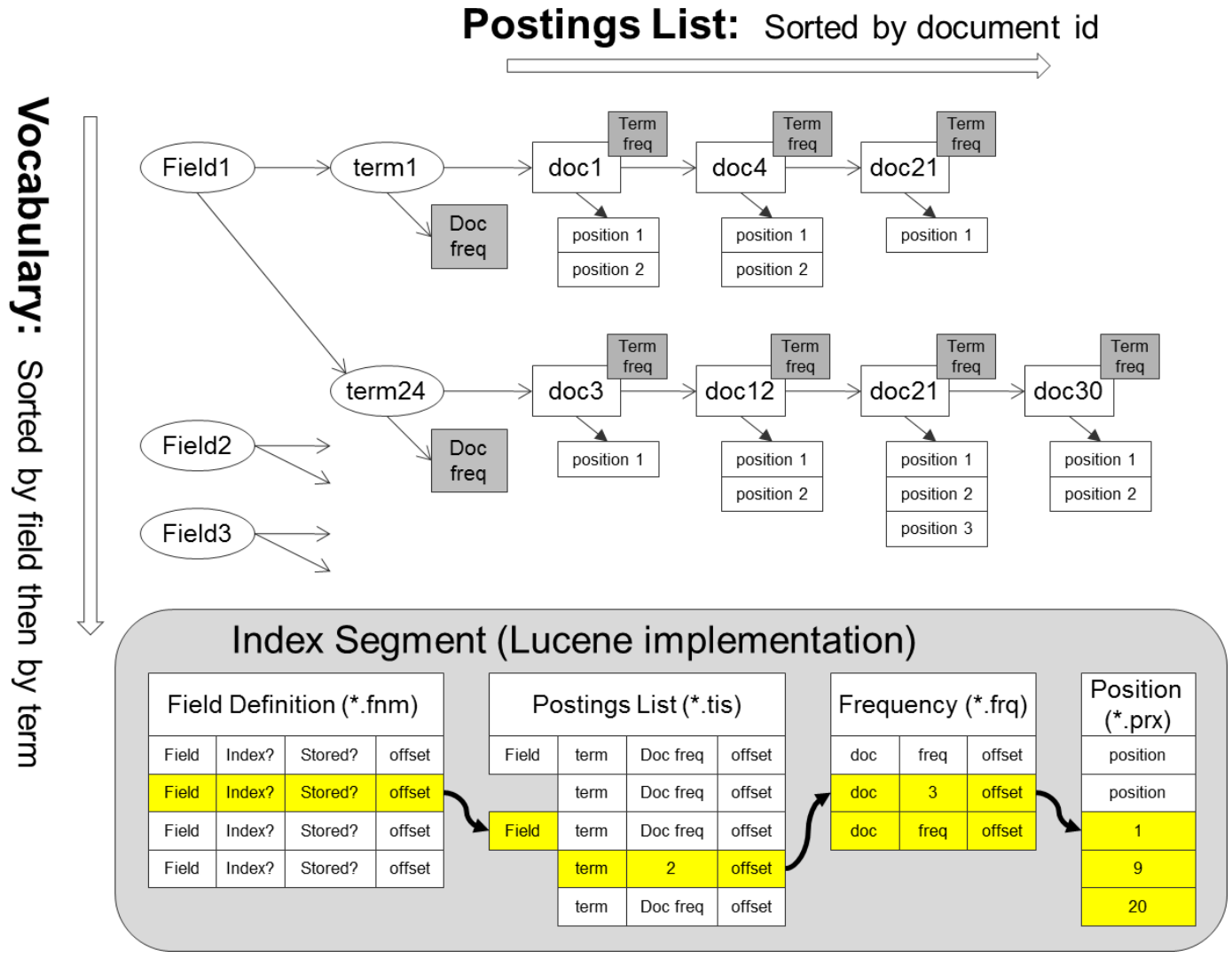




Apache Lucene Indexing Process

- Indexed fields can be
  - Provided in plain text, in which case it will be first passed through the text analysis pipeline to obtain token streams
  - The final form of a sequence of tokens with attributes (i.e., token streams)
- Token streams
  - Are then inverted and added to in-memory segments, which are periodically merged and flushed to disk
  - Depending on the field options, various token attributes (such as positions, starting / ending offsets and per-position payloads) are also stored with the inverted data.
  - It is possible to omit positional information while still storing the in-document term frequencies on a per-field basis
  - Use the Finite State Transducer for their in-memory representation

Apache Lucene Indexing Process





Apache Lucene Indexing Process

- A variant of an indexed field is a field where the creation and storage of term frequency vectors is requested
  - In this case the token stream is used also for building a small inverted index consisting of data from the current field only
  - This inverted data is then stored on a per-document and per-field basis
  - Term frequency vectors are particularly useful when performing document highlighting, relevance feedback or when generating search result snippets (region of text that best matches the query terms)

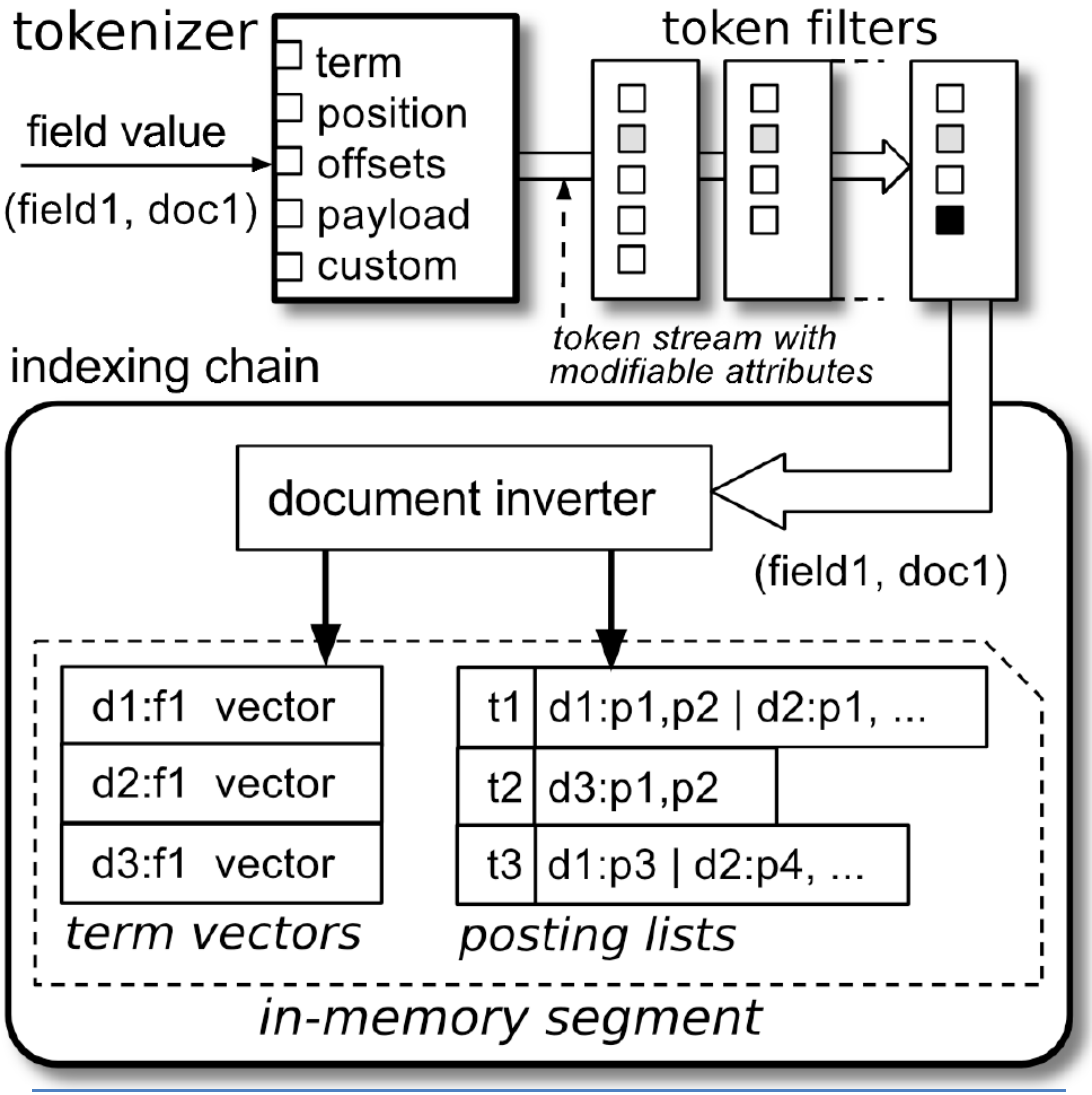


Apache Lucene Indexing Process

- Stored fields are typically used for storing auxiliary per-document data that is not searchable but would be cumbersome to obtain otherwise (e.g., it would require retrieval from a separate system)

- The per-document data is stored as byte arrays
  - Can be manipulated to be encoded using UTF-8 strings, numbers, arrays etc.
  - Optionally, it can be stored using strongly typed API (so called “doc values”) that can use a more optimized storage format, i.e., strongly typed storage
  - This kind of strongly typed storage is used for example to store per-document and per-field weights (so called “norms”, as they typically correspond to field length normalization factor that affects scoring)

Apache Lucene Indexing Process





Apache Lucene Indexing Process

- Indexing Chain
  - The resulting token stream
    - Is finally processed by the indexing chain
    - The supported attributes (term value, position, offsets and payload data) are added to the respective posting lists for each term
    - Posting list - a set of document unique indentifiers (ids) and offsets associated with each term in an inverted index
  - Term values
    - Can be arbitrary byte array values as terms
    - Can use custom comparators to define the sorting order of such terms
  - At this stage documents are assigned their internal document identifiers, i.e., small sequential integers (for efficient delta compression)
  - These identifiers
    - Are used for identifying document data within a particular segment
    - Change after two or more segments are merged (during index compaction)



Apache Lucene Indexing Process

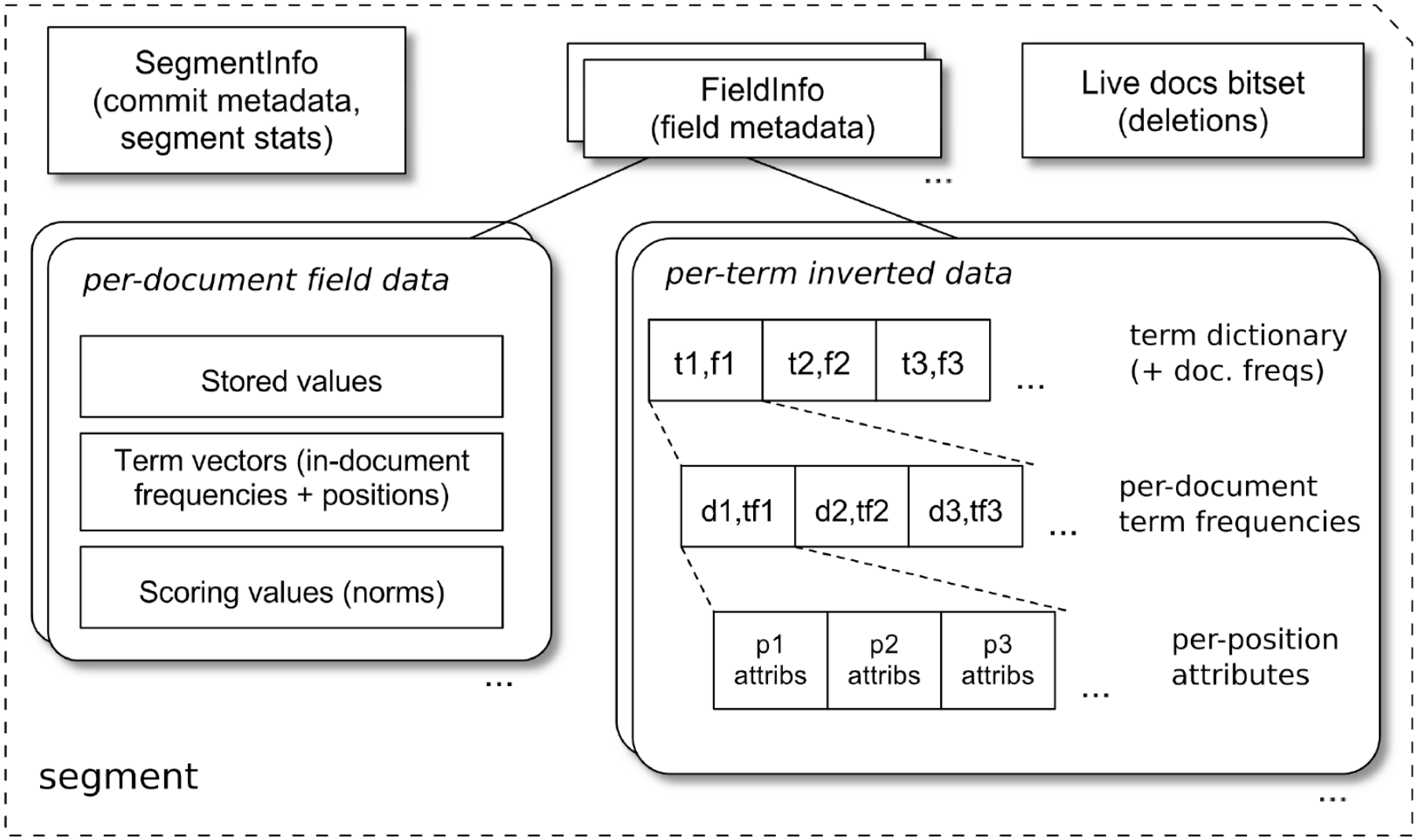
- Incremental Index Updates
  - Indexes can be updated incrementally on-line
    - Simultaneously with searching
    - By adding new documents and/or deleting existing ones (sub-document updates are a work in progress)
  - Index extents
    - Are a common way to implement incremental index updates
    - Do not require modifying the existing parts of the index



Apache Lucene Indexing Process

- When new documents are submitted for indexing
  - It passes through the indexing chain
  - The resulting inverted and non-inverted data is accumulated in new in-memory index extents called segments
  - The segment use a compact in-memory representation (a variant of Codec)
  - Periodically, these in-memory segments are flushed to a persistent storage (using the Codec API and Directory API abstractions)
  - The flushing appears whenever a configurable threshold is reached, e.g., the total number of documents, the size in bytes of the segment, etc.

Apache Lucene Segment





Apache Lucene Indexing Process

- The Indexer Writer
  - The Index Writer is a high-level (Java) class responsible for
    - Processing index updates (additions and deletions)
    - Recording them in new segments and creating new commit points
    - Triggering the index compaction (segment merging)
  - It uses a pool of DocumentWriter-s that create new in-memory segments
  - As new documents are being added and in-memory segments are being flushed to storage, periodically an index compaction (merging) is executed in the background that reduces the total number of segments that comprise the whole index



Apache Lucene Indexing Process

- The Indexer Writer
  - Document deletions
    - Are Queries that select the documents to be deleted
    - Are accumulated:
      - Applied to the in-memory segments before flushing (while they are still mutable)
      - Recorded in a commit point so that they can be resolved when reading the already flushed immutable segments



Apache Lucene Indexing Process

- The Indexer Writer
  - The commit point:
    - Is created at each flush operation or index compaction
    - Is recorded in a global index structure using a two-phase commit
    - Is a list of segments and deletions comprising the whole index at the point in time when the commit operation was successfully completed
  - At the commit point, the segment data that is being flushed from in-memory segments is encoded using the configured Codec implementation



Apache Lucene Indexing Process

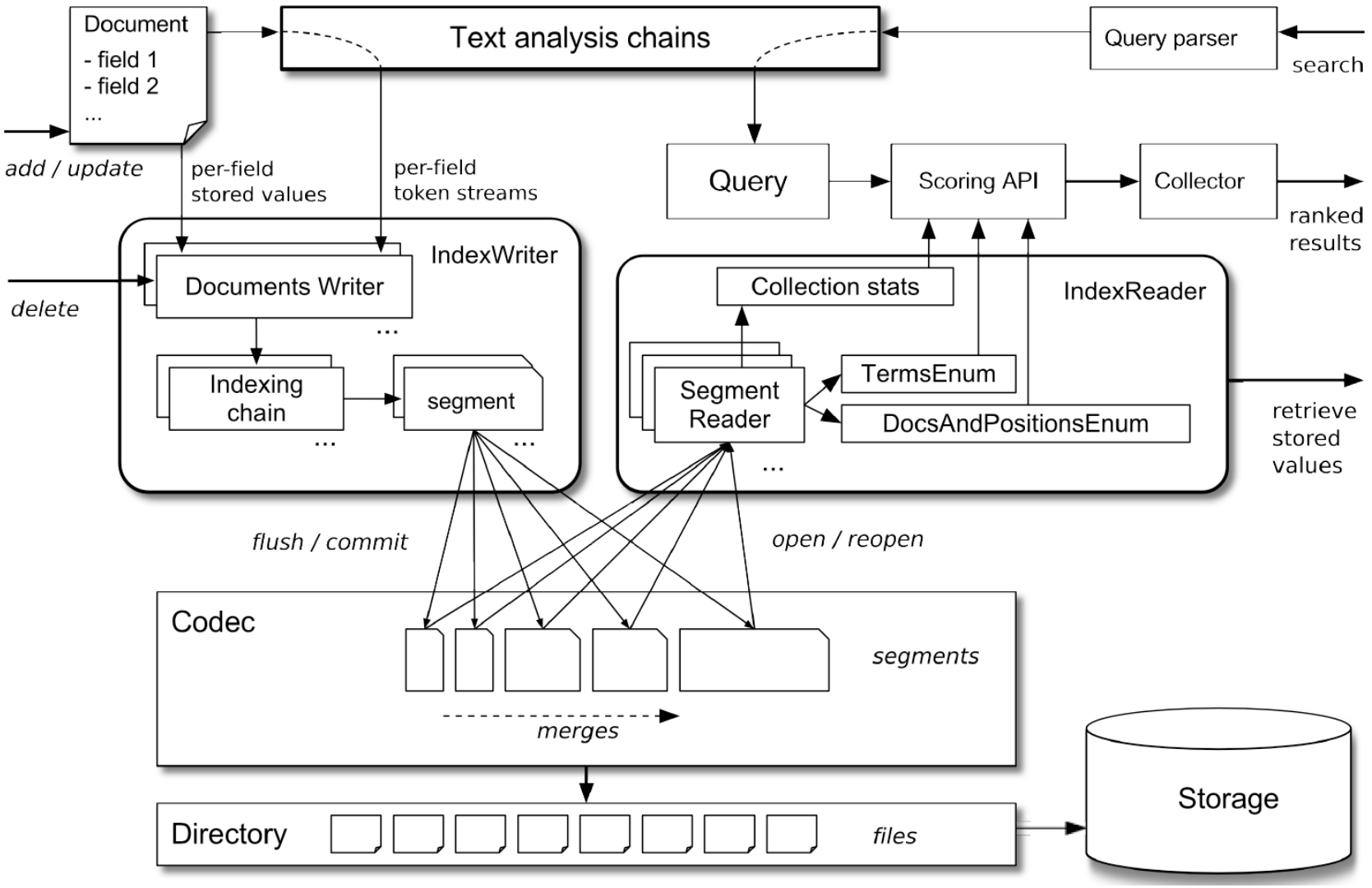
- The Indexer Writer
  - The segment data
    - Are fully immutable (write-once)
    - Any changes are expressed either as
      - New segments
      - New lists of deletions
    - Both types of changes create new commit points
  - The updated view of the latest version of the index becomes visible when a commit point is recorded using a two-phase commit
    - This enables lock-free reading operations concurrently with updates
    - Point-in-time travel by opening the index for reading using some existing past commit point



Apache Lucene Indexing Process

- As segments are immutable
  - Deletions do not actually remove data from existing segments
  - Instead, the delete operations are resolved when existing segments are open
  - Thus, the deletions are represented as a bitset of live (not deleted) documents
  - This bitset is then used when enumerating posting lists and stored fields and during search to hide deleted documents.
  - Global index statistics are not recalculated, so they are slightly wrong (still include term statistics from deleted documents)
  - For performance reasons the data of deleted documents is actually removed only during segment merging and the global index statistics are recalculated

Apache Lucene Querying



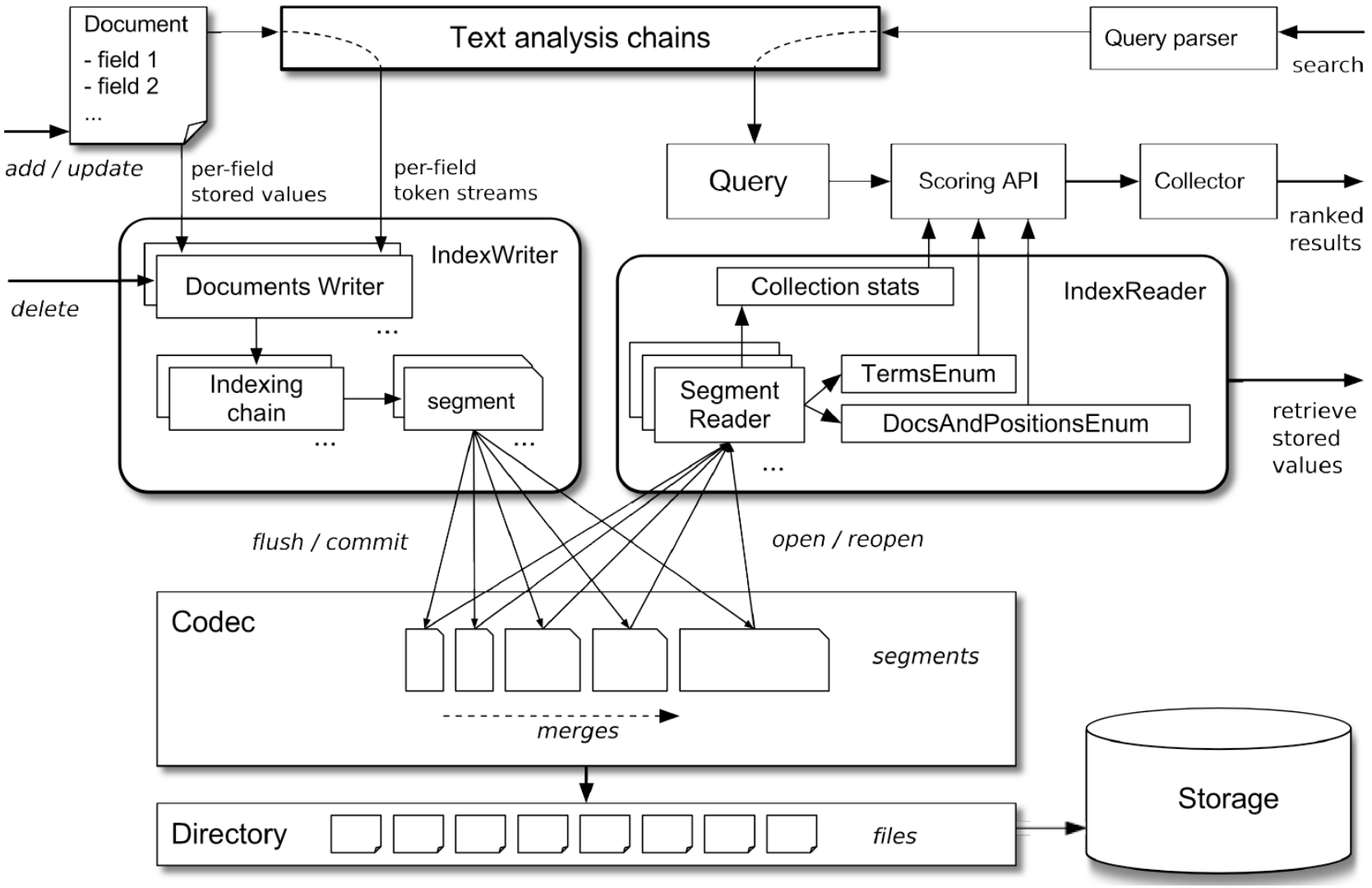


Apache Lucene
Querying

- The Indexer Reader
  - Provides high-level methods to retrieve stored fields, term vectors, and to traverse the inverted index data
  - Uses the Codec API to retrieve and decode the index data
  - Represents the view of an index at a specific point in time

- A user obtains information using an Index Reader from
  - A commit point when all data has been written to disk
  - Directly from Index Writer: a near real-time snapshot that includes both the flushed and the in-memory segments

Apache Lucene Codec API

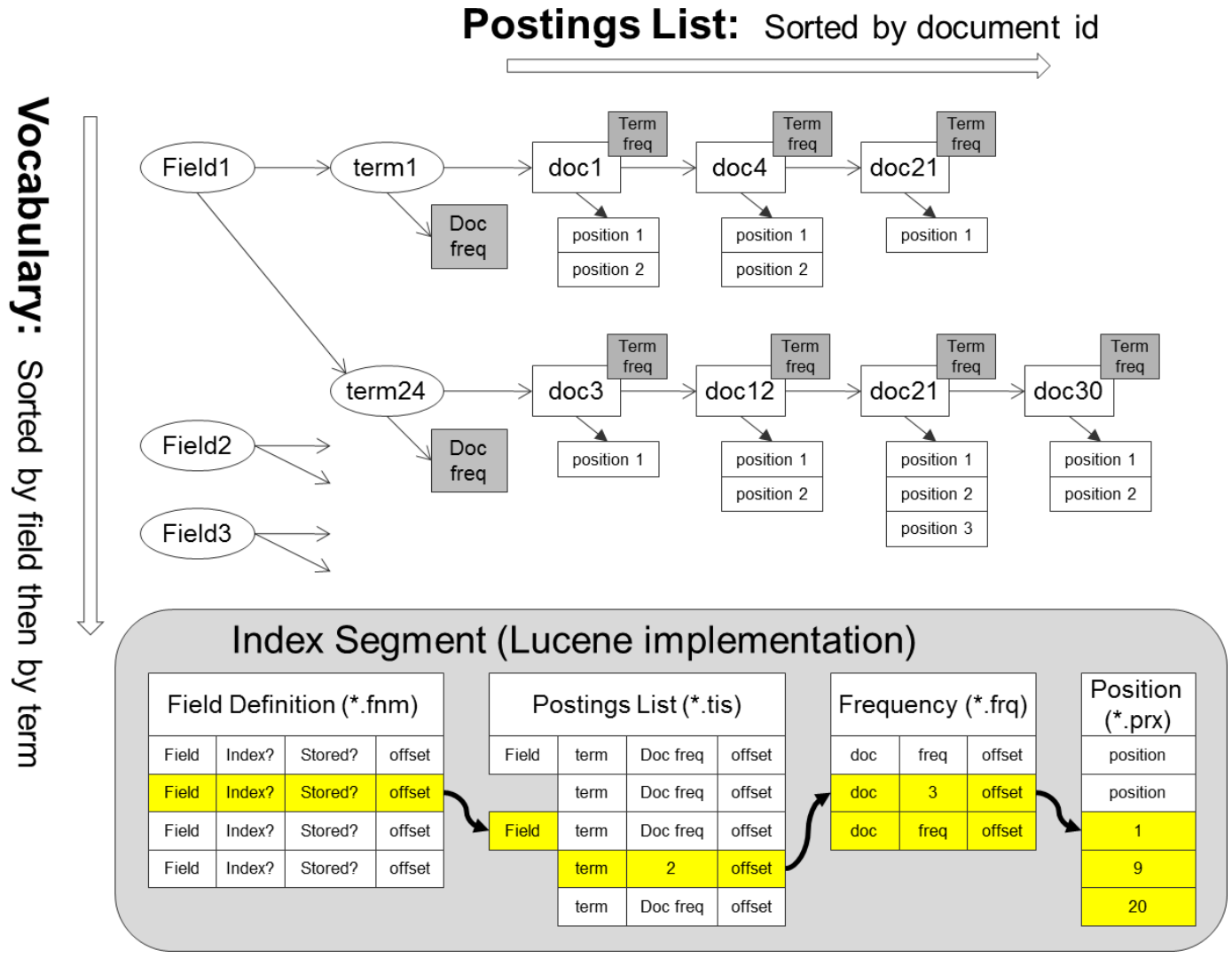




Apache Lucene Codec API

- The Codec API deals with
  - Index data encoding for writing in the storage
  - Index data decoding for reading
  - Segment data merging
- A 4-D View of the Inverted Index
  - The Codec API encapsulates the inverted index data as a logical 4-dimensional table that can be traversed using enumerators
  - The dimensions are: field, term, document, and position
  - Using a cursor, we can advance along rows and columns of this table in each dimension

Apache Lucene Indexing Process

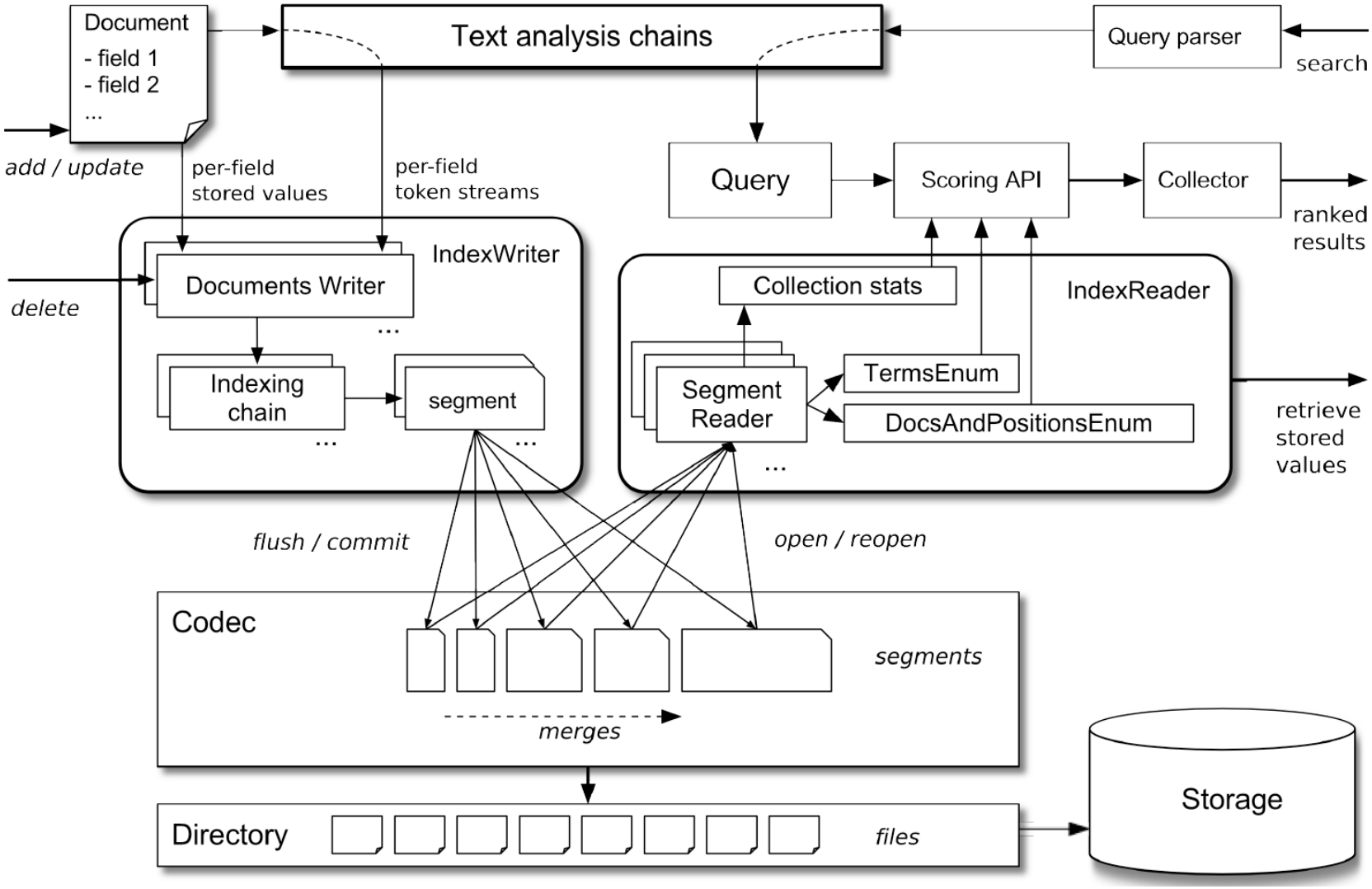




Apache Lucene Codec API

- Lucene Codecs API
  - The default codec implementation uses a combination of well-known compression algorithms and strategies selected to provide a good tradeoff between index size (and related costs of I/O seeks) and coding costs
  - The codes offer good balance between high performance indexing and fast execution of queries.

Apache Lucene Directory

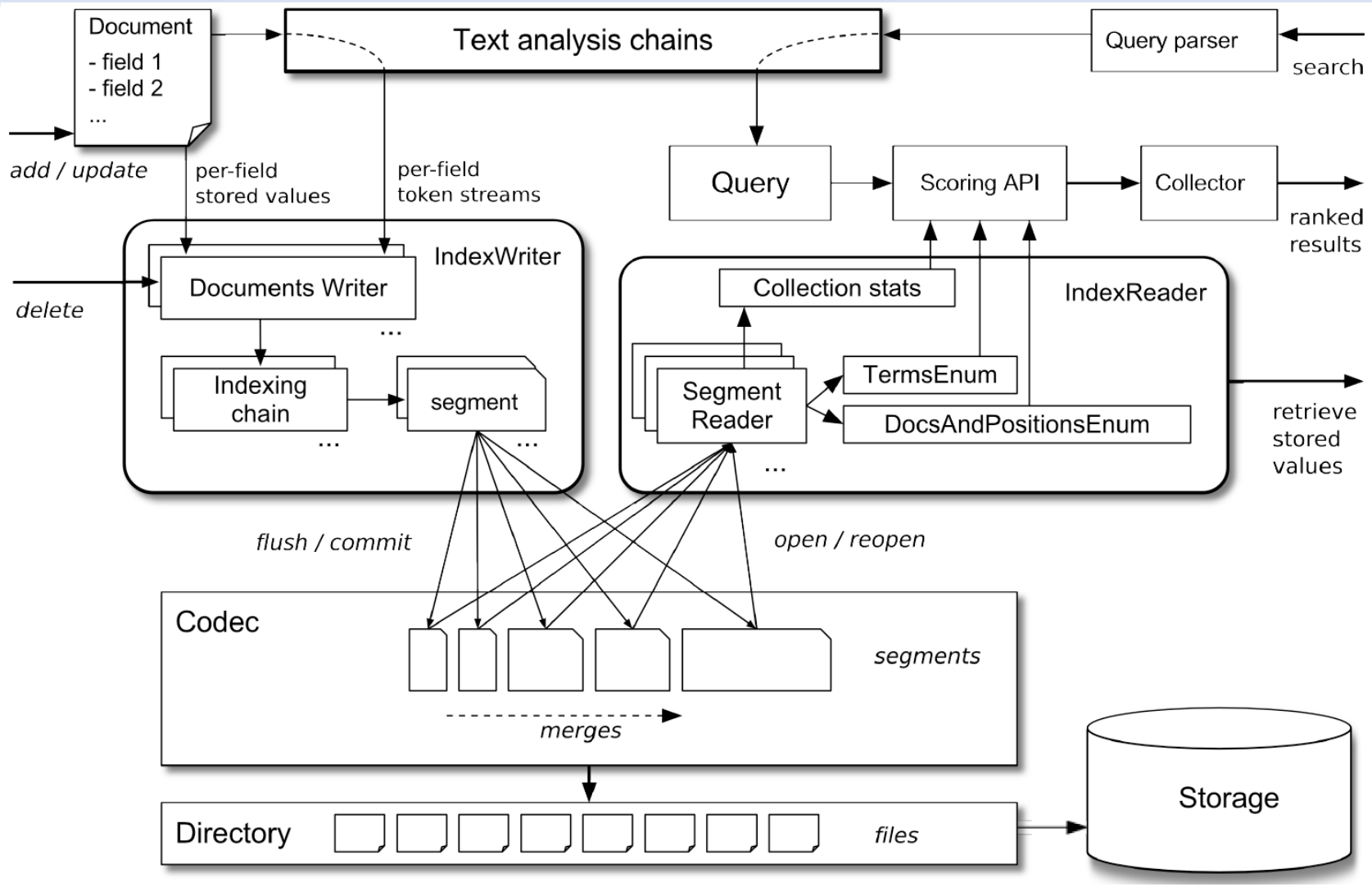




Apache Lucene
Directory API

- Directory API
  - The physical I/O access is abstracted using the Directory API
  - The Directory API offers a very simple file system-like views of the persistent storage
  - The Lucene Directory is basically a flat list of files
  - Files are write-once
  - Abstractions are provided for sequential and random access for writing and reading of files

Apache Lucene Searching





Apache Lucene
Searching

- Searching can be broken down as follows
  - Query model
  - Query types
  - Query evaluation
  - Scoring
  - Common search extensions



Apache Lucene
Query Model

- Query model
  - Lucene does not enforce a particular query language
  - Query objects are used to perform searches
  - Several Queries are provided as building blocks to express complex queries
  - Developers can construct their own query language programmatically or via a Query Parser.



Apache Lucene
Query Types

- Query types used in Lucene:
  - Term queries that evaluate a single term in a specific field
  - Boolean queries
  - Proximity queries
  - Position-based queries that allow to express more complex rules for proximity and relative positions of terms
  - Wildcard, fuzzy and regular expression queries that use automata for evaluating matching terms
  - Disjunction-max query that assigns scores based on the best match for a document across several fields
  - Payload query that processes per-position payload data, etc.



Apache Lucene
Query Functions & Search Queries

- Function queries
  - Lucene also supports the incorporation of field values into scoring, i.e., function queries
  - Function queries can be used to add useful scoring factors like time and distance into the scoring model

- Search query are either
  - Parsed by a Query Parser into a Query tree
  - Generated and combined programmatically



Apache Lucene
Query Evaluation

- Query evaluation
  - At Query execution time, the Scoring API
    - Processed each inverted index segment sequentially for efficiency
    - Calculates a score for each document
  - Documents are scored using one of two strategies:
    - Document-at-a-Time (DAAT)
    - TAAT (Term-at-a-Time) only when the number of terms is low.
  - A Collector is responsible for actually consuming these Scorers and doing something with the results, for example populating a priority queue of the top-N documents



Apache Lucene
Scoring

- Scoring is done using a Similarity class that implements a policy for scoring terms and queries

- When computing a score, the Similarity class considers
  - Term and global index statistics
  - Query specifics (e.g., distance between terms of a phrase, Levenshtein edit distance of fuzzy terms, etc.)

- Lucene scoring uses well-known scoring models: TFIDF, BM25, etc.



Apache Lucene
Common Search Extensions

- Common Search Extensions
  - Lucene provides extended query processing capabilities to support easier navigation of search results:
    - Rollup/drilldown capabilities using the Faceting module
    - Result grouping supports folding related documents into a single combined result.
    - Support for nested documents, query expansion, and geospatial search

References

- A. Białecki, R. Muir, G. Ingersoll. Apache Lucene 4. SIGIR 2012 workshop on open source information retrieval, p. 17. 2012.
- M. McCandless, E. Hatcher, O. Gospodnetic. Lucene in Action (2nd Edition). Maning. 2010
- A. Sharma. Practical Apache Lucene 8. Apress. 2020
- Apache Lucene Documentation https://lucene.apache.org/core/10_1_0/index.html